Лабораторна робота №6.
Реалізація комп'ютерної програми мовою програмування Python з використанням операторів введення та виведення

Мета роботи: ознайомитися з базовими принципами програмування на Python. Навчитися використовувати оператори введення та виведення для створення інтерактивних програм.

Завдання: Кожен індивідуально обирає зі списку запропонованих тематик і реалізовує просту програму з використанням базових операторів у мові програмування Python.

Опис роботи операторів у Python:

Введення/Виведення. `input()` дозволяє користувачу вводити дані, а `print()` виводить інформацію на екран.

```
product = input("Введіть номер товару для додавання в кошик: ")
print(f"{product} додано до кошика.")
```

Словники (`dict`) зберігає дані як ключі-значення з категоріями та текстами.

```
blog = {"Технології": ["Мій перший блог"]}
```

Умови (`if-elif-else`) використовується для перевірки, чи існує рецепт з певним інгредієнтом або в категорії.

```
if "буряк" in ingridients:
    print("Рецепти з буряком:")
```

Цикли (`while`, `for`). `while` використовується для безперервного прийняття команд, `for` для ітерації по статтях.

```
for article in articles:
    print(article)
```

Опис роботи програми:

Програма має попередньо заданий словник користувачів, де ключами є імена користувачів, а значеннями — відповідні паролі.
Функція авторизації:

- **Цикл** `while True`: Програма запускає цикл, що повторюється доти, доки користувач не авторизується або не відмовиться від подальших спроб.
- **Введення імені користувача**: Користувач вводить своє ім'я. Програма перевіряє, чи є це ім'я в словнику.
- **Перевірка пароля**: Якщо ім'я користувача знайдено, програма запитує пароль. Якщо пароль правильний, цикл завершується, і користувач отримує повідомлення про успішну авторизацію.

- **Повторна спроба**: Якщо ім’я користувача або пароль невірні, користувач може спробувати знову або завершити спроби.

Пропозиція завершити спроби: Якщо авторизація не вдалася, програма пропонує користувачу вирішити, чи хоче він спробувати ще раз.

```
# Заздалегідь визначений словник користувачів з паролями
users = {
    "user1": "password1",
    "user2": "password2",
    "user3": "password3"
}

def authorize_user():
    while True:
        # Введення імені користувача
        username = input("Введіть ваше ім'я користувача: ")

        # Перевірка наявності користувача в системі
        if username in users:
            # Введення пароля
            password = input("Введіть ваш пароль: ")

            # Перевірка пароля
            if users[username] == password:
                print(f"Вітаємо, {username}! Ви успішно авторизувалися.")
                break
            else:
                print("Неправильний пароль. Спробуйте ще раз.")
        else:
            print("Ім'я користувача не знайдено. Спробуйте ще раз.")

        # Пропозиція вийти з циклу
        retry = input("Бажаєте спробувати знову? (так/ні): ").strip().lower()
        if retry != 'так':
            print("Авторизація скасована.")
            break

# Виклик функції авторизації
authorize_user()
```

Приклад виконання програми:

```
Введіть ваше ім'я користувача: user1
Введіть ваш пароль: wrongpassword
Неправильний пароль. Спробуйте ще раз.
Бажаєте спробувати знову? (так/ні): так
Введіть ваше ім'я користувача: user1
Введіть ваш пароль: password1
Вітаємо, user1! Ви успішно авторизувалися.
```

1. **Реалізація програми:**
    - Використовуючи оператори введення (`input()`) та виведення (`print()`), умови (`if-elif-else`), циклі (`for або while`) реалізуйте обране завдання (з 8 запропонованих у списку нижче).
    - Програма повинна коректно обробляти введені користувачем дані та виводити результати.
2. **Тестування:**
    - Перевірте програму на різних вхідних даних, щоб упевнитися в її правильності та надійності.
    - Додайте обробку можливих помилок (наприклад, неправильний формат введених даних).
3. **Додаткове завдання (за бажанням):**
    - Додайте до програми можливість виконання кількох операцій поспіль, не виходячи з програми. Реалізуйте цикл, який дозволяє користувачу вибирати інші операції або завершити програму.

3. Оформлення звіту:

- У звіті опишіть реалізовану програму, включаючи її код та приклади роботи.
- Додайте скріншоти з прикладами виконання програми.
- Поясніть, як ви вирішували можливі проблеми, що виникали під час розробки.

Вибір завдання:

1. Список завдань

Вивід програми:

```
1. Додати завдання
2. Переглянути завдання
3. Відредагувати завдання
4. Видалити завдання
5. Позначити як виконане
6. Вийти
Виберіть дію: 1
Введіть назву завдання: Прочитати книгу

1. Додати завдання
2. Переглянути завдання
3. Відредагувати завдання
4. Видалити завдання
5. Позначити як виконане
6. Вийти
Виберіть дію: 1
Введіть назву завдання: Написати звіт

1. Додати завдання
2. Переглянути завдання
3. Відредагувати завдання
4. Видалити завдання
```

```
5. Позначити як виконане
6. Вийти
Виберіть дію: 2
1. Прочитати книгу [✗]
2. Написати звіт [✗]

Виберіть дію: 5
Введіть номер завдання для позначення як виконане: 1

Виберіть дію: 2
1. Прочитати книгу [✓]
2. Написати звіт [✗]

Виберіть дію: 6
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`, `for`
- **Списки**: `append()`, `pop()`

---

2. Калькулятор

Вивід програми:

```
Введіть перше число: 10
Введіть операцію (+, -, *, /): +
Введіть друге число: 5
Результат: 15.0
Виконати ще одну операцію? (так/ні): так

Введіть перше число: 10
Введіть операцію (+, -, *, /): /
Введіть друге число: 2
Результат: 5.0
Виконати ще одну операцію? (так/ні): ні
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`
- **Арифметичні операції**: `+`, `-`, `*`, `/`

---

3. Конвертер одиниць

Вивід програми:

```
1. Кілометри в милі
2. Мили в кілометри
3. Кілограми в фунти
4. Фунти в кілограми
5. Вийти
Виберіть тип конвертації: 1
Введіть кількість кілометрів: 10
10 км = 6.21371 миль

Виберіть тип конвертації: 3
Введіть кількість кілограмів: 70
70 кг = 154.3234 фунтів

Виберіть тип конвертації: 5
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`
- **Арифметичні операції**: множення, ділення

---

4. Система голосування

Вивід програми:

```
Голосування:
- Варіант 1
- Варіант 2
- Варіант 3
Виберіть варіант: Варіант 1

Голосувати ще раз? (так/ні): так

Голосування:
- Варіант 1
- Варіант 2
- Варіант 3
Виберіть варіант: Варіант 3

Голосувати ще раз? (так/ні): так

Голосування:
- Варіант 1
- Варіант 2
- Варіант 3
Виберіть варіант: Варіант 2
```

```
Голосувати ще раз? (так/ні): ні

Результати голосування:
Варіант 1: 1 голосів
Варіант 2: 1 голосів
Варіант 3: 1 голосів
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-else`
- **Цикли**: `while`, `for`
- **Словники**: `votes` для підрахунку голосів

5. Простий магазин

Вивід програми:

```
1. Переглянути товари
2. Додати товар до кошика
3. Переглянути кошик
4. Оформити замовлення
5. Вийти
Виберіть дію: 1
Товари в магазині:
1. Книга - 150 грн
2. Ручка - 20 грн
3. Ноутбук - 15000 грн

Виберіть дію: 2
Введіть номер товару для додавання в кошик: 1
Книга додана до кошика.

Виберіть дію: 3
Товари в кошику:
1. Книга - 150 грн

Виберіть дію: 4
Замовлення оформлено. Загальна сума: 150 грн.

Виберіть дію: 5
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`, `for`
- **Списки**: для збереження товарів та кошика

---

6. Персональний блог

Вивід програми:

```
1. Створити нову статтю
2. Переглянути всі статті
3. Переглянути статті за категоріями
4. Вийти
Виберіть дію: 1
Введіть заголовок статті: Мій перший блог
Введіть категорію: Технології
Введіть текст статті: Це мій перший запис у блозі.

Стаття додана!

Виберіть дію: 2
Список статей:
1. Мій перший блог (Категорія: Технології)

Виберіть дію: 3
Введіть категорію для перегляду: Технології
Статті у категорії "Технології":
1. Мій перший блог

Виберіть дію: 4
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`, `for`
- **Словники**: для зберігання статей та їх категорій

---

7. Галерея зображень

Вивід програми:

```
1. Завантажити зображення
2. Переглянути всі зображення
3. Переглянути зображення за категоріями
4. Пошук зображення за назвою
5. Вийти
Виберіть дію: 1
Введіть назву зображення: Літо
Введіть категорію: Природа
Зображення "Літо" додано до категорії "Природа".

Виберіть дію: 2
Список всіх зображень:
1. Літо (Категорія: Природа)
```

```
Виберіть дію: 3
Введіть категорію для перегляду: Природа
Зображення у категорії "Природа":
1. Літо

Виберіть дію: 4
Введіть назву зображення для пошуку: Літо
Зображення знайдено: Літо (Категорія: Природа)

Виберіть дію: 5
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`, `for`
- **Словники**: для зберігання зображень та їх категорій

---

8. Книга рецептів

Вивід програми:

```
1. Додати рецепт
2. Переглянути всі рецепти
3. Пошук рецепту за інгредієнтом
4. Переглянути рецепти за категоріями
5. Вийти
Виберіть дію: 1
Введіть назву рецепту: Борщ
Введіть інгредієнти через кому: буряк, картопля, капуста
Введіть категорію: Українська кухня
Рецепт "Борщ" додано.

Виберіть дію: 2
Список всіх рецептів:
1. Борщ (Категорія: Українська кухня)

Виберіть дію: 3
Введіть інгредієнт для пошуку: буряк
Рецепти з інгредієнтом "буряк":
1. Борщ

Виберіть дію: 4
Введіть категорію для перегляду: Українська кухня
Рецепти у категорії "Українська кухня":
1. Борщ

Виберіть дію: 5
```

Використані оператори:

- **Введення/Виведення**: `input()`, `print()`
- **Умови**: `if-elif-else`
- **Цикли**: `while`, `for`
- **Словники та списки**: для зберігання рецептів, інгредієнтів та категорій.